

What a prediction model figures out about its test questions
which numbers it recovers, how fast, and why

Wenrui Yuan

Prediction is not the same as measurement

We train a model to do one job, predict whether a learner answers the next question correctly. Answers are right or wrong, or graded into a few levels.

To predict well, the model works out three numbers on its own.

- **score**, how strong a learner is.
- **difficulty**, how hard a question is.
- **sharpness**, how clearly a question tells apart strong and weak learners.

We never supply these numbers and we never add a target for them. The model is graded only on its answer predictions. The research question is whether good prediction also gives back faithful measurement, and that is not guaranteed.

What we measure, and how

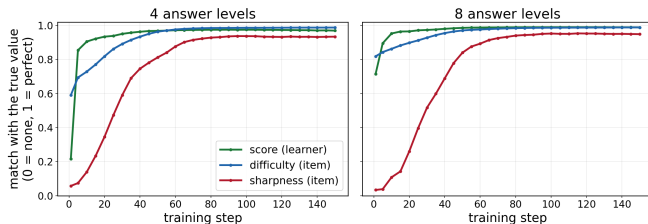
On made-up data the three true numbers are known, so we can score the model's recovered numbers against them.

- We report **match in order** (rank agreement, 0 to 1), not raw values. The scale and zero point of these numbers are not fixed by the data, only their pattern is, so we compare order rather than raw values.
- We track recovery **over training**, not just at the end, because the path matters as much as the endpoint.
- Every claim below carries a **control**, a capacity match, a negative control, a matched starting point, or a matched null, stated on each slide.

Four findings follow. Each is one observation, one hypothesis, one designed test, one result.

Finding 1, observation and hypothesis

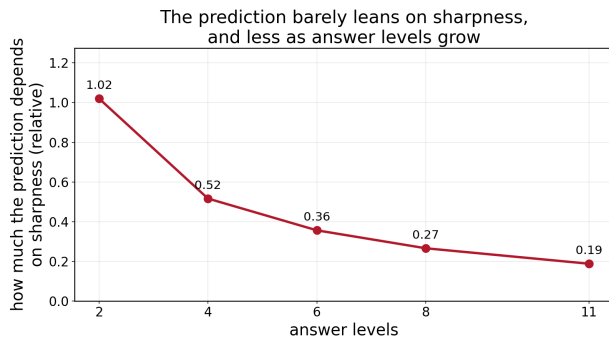
Score and difficulty are learned fast; sharpness lags



Observation. Score and difficulty reach a near perfect match within the first few to twenty training steps. Sharpness rises slowly and settles a notch lower (about 0.93 against 0.97 and 0.99). It is consistently the last of the three to come in.

Hypothesis. Sharpness lags because the prediction barely depends on it. A model can only pin down a number through that number's effect on the predicted answer, so a number the prediction is insensitive to is learned slowly.

Finding 1, design and result



Design. Measure how much the predicted answer moves when each number is nudged (its sensitivity, the Fisher information), and compare that ordering to the recovery speeds. Repeat across answer levels.

Result. The prediction leans heavily on score and difficulty and weakly on sharpness. For a learner at a question's level the odds stay near a coin flip whatever its sharpness, and far above or below the answer is near certain either way, so only the band between is sensitive. The overall pull is small and shrinks with answer levels, from about 1.0 at two levels to 0.19 at eleven.

Conclusion. A number's leverage on the prediction governs how fast and how well it is recovered. Sharpness is the low-leverage number.

Finding 2, hypothesis and design

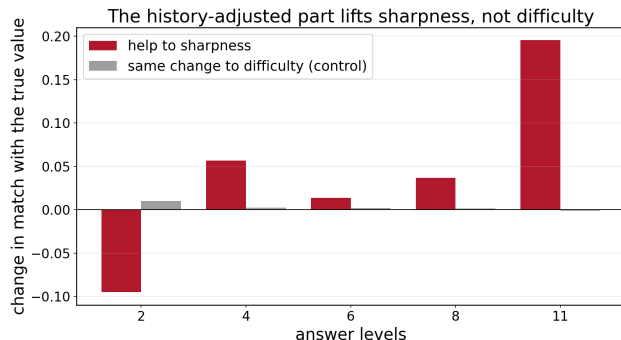
Hypothesis. If sharpness is starved of signal, give it a richer readout. Let each question's sharpness be a fixed part for the question plus a small extra part that shifts with what the learner has answered so far.

Design. Four model variants, holding everything else equal.

- fixed sharpness, narrow, the baseline.
- fixed sharpness, wide, a **capacity-matched control** (same added size, no history).
- fixed plus history sharpness, the variant under test.
- fixed plus history applied to **difficulty**, a **negative control** on a high-leverage number.

We compare recovery across answer levels and over training, not at the endpoint alone.

Finding 2, result



Result. The history part lifts sharpness recovery, and the lift grows with answer levels, up to about +0.20 at eleven. The same change to difficulty does almost nothing, and the capacity-matched wide baseline does not close the gap where the history part helps (it pays in score recovery instead), so the gain is specific, not just added size.

Two honest bounds. The benefit is in speed and reliability, both variants tie if trained to convergence, and the recovered quantity is the order of sharpness, not its size. At two answer levels the extra part slightly hurts.

Conclusion. A targeted readout helps the low-leverage number recover faster and more reliably, more so as the task gets richer, without an endpoint cost.

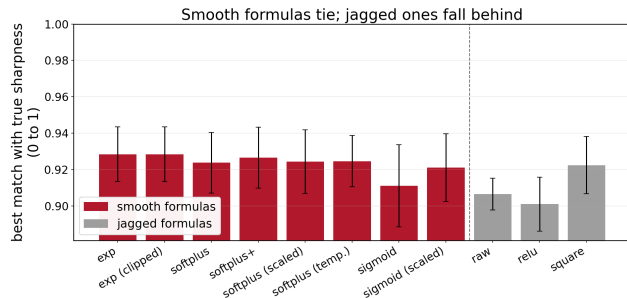
Finding 3, hypothesis and design

Hypothesis. Sharpness must stay positive, so it is read through an always-positive formula. A natural guess is that one particular choice, the exponential, is what makes sharpness learnable, because of how it rescales the updates during training.

Design. A fair contest between always-positive formulas (exponential, softplus and its variants, sigmoid, and the cruder raw, relu, square).

- **Matched starting point**, every formula begins at the same effective sharpness, so none is handed a head start.
- **Per-formula learning-rate tuning**, so no formula simply had a better rate.
- A **direct test** of the training-step idea, optimizing sharpness alone with the other two numbers fixed to truth, under each step rule.

Finding 3, result



Result. Under the fair contest, all the smooth formulas tie (within about 0.02). Only the crude ones lag, the one that can still slip negative and the one that flattens to zero and stops responding. The direct test of the training-step idea also comes up empty, the step rules converge to the same recovery.

Conclusion. The exponential is not special. Staying positive matters and being smooth matters, the particular formula does not. The hypothesis is ruled out, a distractor removed. The direct test shows the positive formula does not reshape the training in any decisive way. The benefit instead traces to the dedicated readout of Finding 2.

Finding 4, design, result, and limit

Design. Plant a real, known link between a question's sharpness and the learner's strength, then ask the history part to recover it. Compare against a matched control with no planted link, so any baseline structure is subtracted.

Result.

- It recovers the **direction** of the planted link.
- It recovers the **size** only weakly, about 4 percent of the true effect, a roughly 25-fold shrinkage.
- It shows a little structure even when nothing is planted, so a small adjustment can be noise.

Conclusion. Read the history part for direction, not for amount. This matches Finding 1, the prediction has too little leverage on sharpness to pin its magnitude, only its ordering.

Why the low-leverage number is the slow one

The four findings share one cause, set out in the supporting analysis.

- The prediction's sensitivity to sharpness works out to the learner-question gap times the answer's uncertainty, (score minus difficulty) times $p(1 - p)$. It is zero when the learner sits at the question's level, and zero far away, so its overall pull is small and shrinks as answer levels grow.
- That weak signal sets the **rate** at which sharpness is learned, not a permanent error. With unlimited data and training the gap closes, which is why the fix is a speed gain, not an endpoint gain.
- The same weak signal is why magnitude is not recovered, only order.

This is a modest, local account that explains the pattern. It is the support, not the headline. The headline is the measured behavior.

Takeaway

One line

A model trained only to predict answers learns each question number only as well as the prediction leans on it.

- Score and difficulty are easy. Sharpness is the low-leverage number, learned slowly and only in order.
- A fixed plus history readout helps sharpness recover faster and more reliably, more so with more answer levels, with no endpoint cost. It gives the order of sharpness, not its size.
- We tested and ruled out the idea that a special positive formula was the key. Staying positive and smooth matters, the formula does not.

Scope. Made-up data, three to ten repeats, one model family. **Next.** A check on real learner data, where the true numbers are unknown and we test stability instead of recovery.